FSBM
FACULTE DES SCIENCES BEN M'SIK
UNIVERSITÉ HASSAN II DE CASABLANCA

MATHÉMATIQUES & INFORMATIQUE

UNIVERSITE HASSAN II DE CASABLANCA

Rapport de projet fin de module

Présenté par :

Khamalli Nour
El Filali Malak
Chouiekh Majda
El Kassmi Nadia

Filière :

Master d'excellence en Intelligence Artificielle

Sujet :

Mise en place d'une infrastructure DevOps pour une application de réservation sur AWS

Encadré par : Pr.Benabbou Faouzia

Présenté le 24/03/2026

2025 – 2026

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude à mon encadrante, Madame **Pr. Benabbou Faouzia**, pour son accompagnement, sa disponibilité et ses précieux conseils tout au long de la réalisation de ce projet. Son encadrement m'a permis de progresser et d'approfondir mes connaissances dans le domaine du DevOps et de l'orchestration des applications.

Je remercie également l'ensemble des enseignants de la Faculté des Sciences Ben M'Sick pour la qualité de la formation dispensée, qui m'a permis d'acquérir les compétences nécessaires à la réalisation de ce projet.

Mes remerciements s'adressent aussi à toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de ce travail, que ce soit par leur soutien, leurs conseils ou leurs encouragements.

Je souhaite enfin remercier ma famille et mes proches pour leur soutien constant, leur patience et leur encouragement tout au long de mon parcours.

Résumé

Ce projet s'inscrit dans le contexte du Cloud Computing et du DevOps, où l'automatisation et la scalabilité des applications sont devenues essentielles. L'objectif principal est de concevoir et de déployer une application de réservation d'hôtel sur une infrastructure cloud, en utilisant des outils modernes d'automatisation et d'orchestration.

Pour cela, une infrastructure a été mise en place sur AWS à l'aide de Terraform, permettant de créer automatiquement les ressources nécessaires telles que les machines virtuelles et le réseau. Ensuite, Ansible a été utilisé pour configurer les serveurs et installer les composants essentiels, notamment Docker et Kubernetes.

L'application a été conteneurisée à l'aide de Docker, facilitant ainsi son déploiement et sa portabilité. Un pipeline CI/CD basé sur GitHub Actions a été mis en place afin d'automatiser les étapes de build, de test et de déploiement.

Le cluster Kubernetes, composé d'un nœud maître et d'un nœud worker, permet d'orchestrer les conteneurs et d'assurer la disponibilité de l'application. Un tableau de bord Kubernetes a également été déployé pour superviser le cluster, avec un accès sécurisé via un token d'authentification.

Les résultats obtenus montrent un déploiement fonctionnel et automatisé de l'application, démontrant l'efficacité de l'approche DevOps dans la gestion des applications modernes.

Abstract

This project is carried out in the context of Cloud Computing and DevOps, where automation and scalability have become essential for modern applications. The main objective is to design and deploy a hotel reservation application on a cloud infrastructure using modern automation and orchestration tools.

The infrastructure is provisioned on AWS using Terraform, which enables the automated creation of resources such as virtual machines and networking components. Then, Ansible is used to configure the servers and install essential tools, including Docker and Kubernetes.

The application is containerized using Docker, ensuring portability and ease of deployment. A CI/CD pipeline based on GitHub Actions is implemented to automate the build, test, and deployment processes.

The Kubernetes cluster, composed of a master node and a worker node, orchestrates the containers and ensures application availability. A Kubernetes Dashboard is also deployed to monitor the cluster, with secure access through an authentication token.

The results demonstrate a fully functional and automated deployment, highlighting the effectiveness of the DevOps approach in managing modern cloud-based applications.

Table de matières :

Remerciements 1

Résumé 2

Abstract 3

Introduction générale 9

Chapitre 1 : Contexte du projet 10

1.1 Contexte général 10

1.2 Problématique 10

1.3 Objectifs du projet 11

1.4 Besoins du projet 11

1.4.1 Besoins fonctionnels 11

1.4.2 Besoins non fonctionnels 11

Chapitre 2 : Architecture et technologies utilisées 12

2.1 Introduction 12

2.2 Architecture globale du système 12

2.3 Technologies utilisées 13

2.3.1 AWS (Amazon Web Services) 13

2.3.2 Terraform 13

2.3.3 Ansible 14

2.3.4 Docker 14

2.3.5 Kubernetes 14

2.3.6 GitHub Actions (CI/CD) 15

2.4 Sécurité et accès au système 15

2.4.1 Accès SSH 15

2.4.2 Accès au Kubernetes Dashboard 15

2.5 Conclusion 16
Chapitre 3 : Mise en œuvre du projet 17
3.1 Introduction 17
3.2 Mise en place de l'environnement AWS 17
Configuration des accès AWS 17
Rôle d'AWS dans le projet 18
3.3 Mise en place de l'infrastructure avec Terraform 18
Initialisation du projet 18
Planification 19
Création de l'infrastructure 19
3.4 Configuration des serveurs avec Ansible 20
3.5 Accès sécurisé via SSH 23
3.6 Vérification du cluster Kubernetes 23
3.7 Configuration de kubectl (kubeconfig) 23
3.8 Kubernetes Dashboard et authentification 24
3.8.1 Déploiement 24
3.8.2 Création de l'utilisateur admin + récupération du token 25
3.8.3 Accès au Dashboard 25
3.9 Conteneurisation avec Docker 25
3.10 Gestion du code source avec GitHub: 26
3.11 Pipeline CI/CD avec GitHub Actions 28
3.12 Déploiement de l'application 28
3.13 Accès à l'application 29
3.14 Conclusion 30
Chapitre 4 : Présentation et résultats de l'application 31

4.1 Présentation générale de l'application 31

Chapitre 5 : Analyse des résultats et validation du système 36

5.1 Introduction 36

5.2 Résultats du déploiement Kubernetes 36

5.3 Validation du fonctionnement de l'application 36

5.4 Supervision et gestion du système 36

5.5 Analyse des performances 37

5.6 Limites du système 37

5.7 Perspectives d'amélioration 37

5.8 Conclusion 38

Perspectives d'évolution 39

Conclusion générale 40

Références 41

Documentation officielle 41

Articles et ressources utiles 41

Liste des figures

Figure 1 : Architecture globale du système 12

Figure 2 : Clé SSH 15

Figure 3 : token d'authentification 15

Figure 4 : Commande pour accéder au Dashboard 16

Figure 5 : Kubernetes Dashboard 16

Figure 6 : Clé d'accès 17

Figure 7 : Commande d'initialisations de terraform 18

Figure 8 : Commande de Planification 19

Figure 9 : Commande pour la Création de l'infrastructure 19

Figure 10 : Les deux instance créer 20

Figure 11 : Terraform output 20

Figure 12 : Mise à jour de l'inventaire 21

Figure 13 : Test de connectivité 21

Figure 14 : Installation de Docker 22

Figure 15 : Installation de Kubernetes 22

Figure 16 : Initialisation du Master 22

Figure 17 : Ajout du Worker 22

Figure 18 : Accès sécurisé via SSH 23

Figure 19 : Vérification du clusters 23

Figure 20 : Kubeconfig 24

Figure 21 : Déploiement 24

Figure 22 : Token d'authentification 25

Figure 23 : Accès Dashboard 25

Figure 24 : Création de l'image 26

Figure 25 : Gestion des images 26

Figure 26 : L'image dans Docker 26

Figure 27: Clonage du repository GitHub du projet sur la machine locale..... 27

Figure 28 : Structure du repository GitHub contenant les fichiers de l'application, les configurations Docker, Kubernetes et le pipeline CI/CD..... 27

Figure 29: Exécution du pipeline GitHub Actions 28

Figure 30 : État des nodes et des pods du cluster Kubernetes..... 29

Figure 31: Interface de l'application web 29

Figure 32 : Page d'acceuil..... 31

Figure 33 : Présentation des services 32

Figure 34 : Processus de réservation 32

Figure 35 : Formulaire de réservation de chambres 33

Figure 36 : Formulaire de réservation au Restaurant 33

Figure 37: Formulaire de réservation de Salle de réunion 34

Figure 38 : Confirmation de réservation 34

Figure 39 : Stockage des données 35

Figure 40 : Réservation des Clients..... 35

Introduction générale

Au cours des dernières années, le développement des technologies de l'information a profondément transformé la manière dont les applications sont conçues, déployées et exploitées. Avec l'émergence du Cloud Computing, les entreprises disposent désormais d'infrastructures flexibles et évolutives, capables de répondre rapidement aux besoins croissants en termes de performance et de disponibilité.

Parallèlement, l'approche DevOps s'est imposée comme une pratique incontournable visant à améliorer la collaboration entre les équipes de développement et d'exploitation. Elle repose sur l'automatisation des processus, la réduction des délais de mise en production et l'amélioration continue de la qualité des applications. Dans ce cadre, plusieurs outils et technologies ont vu le jour, notamment les solutions de conteneurisation comme Docker, ainsi que les plateformes d'orchestration telles que Kubernetes.

En outre, l'automatisation de l'infrastructure, à travers des outils comme Terraform, et la configuration des systèmes via Ansible, permettent de simplifier la gestion des environnements complexes tout en garantissant leur reproductibilité. De plus, l'intégration de pipelines d'intégration et de déploiement continus (CI/CD), notamment avec GitHub Actions, facilite l'automatisation des différentes étapes du cycle de vie des applications.

Dans ce contexte technologique en constante évolution, les applications web modernes, telles que les systèmes de réservation en ligne, doivent être conçues de manière à garantir leur disponibilité, leur performance et leur facilité de déploiement.

C'est dans cette optique que s'inscrit le présent projet, qui vise à mettre en œuvre une chaîne DevOps complète pour le déploiement d'une application de réservation d'hôtel sur une infrastructure cloud, en s'appuyant sur des technologies modernes telles que AWS, Docker, Kubernetes, Terraform, Ansible et GitHub Actions.

Chapitre 1 : Contexte du projet

1.1 Contexte général

Avec la transformation numérique, les entreprises s'appuient de plus en plus sur des applications web pour offrir leurs services en ligne. Ces applications doivent être capables de gérer un grand nombre d'utilisateurs tout en garantissant une haute disponibilité, une bonne performance et une sécurité adéquate.

Dans ce cadre, le Cloud Computing s'est imposé comme une solution incontournable, permettant de disposer d'infrastructures flexibles, évolutives et accessibles à la demande. Par ailleurs, l'approche DevOps a permis de révolutionner la manière dont les applications sont développées et déployées, en favorisant l'automatisation et la collaboration entre les équipes.

L'émergence de technologies telles que Docker pour la conteneurisation et Kubernetes pour l'orchestration a permis de simplifier le déploiement des applications tout en assurant leur portabilité et leur scalabilité. De plus, l'utilisation d'outils comme Terraform et Ansible permet d'automatiser respectivement la création de l'infrastructure et la configuration des systèmes.

1.2 Problématique

Malgré l'existence de ces technologies, le déploiement d'une application complète reste un processus complexe qui nécessite la coordination de plusieurs outils et étapes.

Les principales problématiques rencontrées sont :

- La difficulté de mettre en place une infrastructure cloud de manière rapide et fiable
- La complexité de la configuration des serveurs
- Le manque d'automatisation dans le processus de déploiement
- Les erreurs humaines lors des manipulations manuelles
- La difficulté de garantir la disponibilité et la scalabilité des applications

Ainsi, il devient nécessaire de mettre en place une solution intégrée permettant d'automatiser l'ensemble du processus, depuis la création de l'infrastructure jusqu'au déploiement de l'application.

1.3 Objectifs du projet

L'objectif principal de ce projet est de concevoir et de déployer une application de réservation d'hôtel en utilisant une approche DevOps complète.

Les objectifs spécifiques sont les suivants :

- Mettre en place une infrastructure cloud sur AWS à l'aide de Terraform
- Automatiser la configuration des serveurs avec Ansible
- Installer et configurer un cluster Kubernetes (Master et Worker)
- Conteneuriser l'application à l'aide de Docker
- Mettre en place un pipeline CI/CD avec GitHub Actions
- Déployer l'application sur Kubernetes
- Superviser le cluster à l'aide du Kubernetes Dashboard

1.4 Besoins du projet

1.4.1 Besoins fonctionnels

Le système doit permettre :

- Le déploiement d'une application de réservation d'hôtel
- L'accès à l'application via un navigateur web
- La gestion des conteneurs via Kubernetes
- La supervision du cluster à travers un dashboard

1.4.2 Besoins non fonctionnels

Le système doit garantir :

- La disponibilité de l'application
- La scalabilité de l'infrastructure
- La sécurité des accès (SSH, token Kubernetes)
- L'automatisation des processus (CI/CD)
- La performance et la rapidité de déploiement

Chapitre 2 : Architecture et technologies utilisées

2.1 Introduction

Dans ce chapitre, nous présentons l'architecture globale du système ainsi que les différentes technologies utilisées pour la réalisation du projet. Cette architecture s'inscrit dans une approche DevOps complète intégrant l'automatisation, la conteneurisation et l'orchestration des applications.

2.2 Architecture globale du système

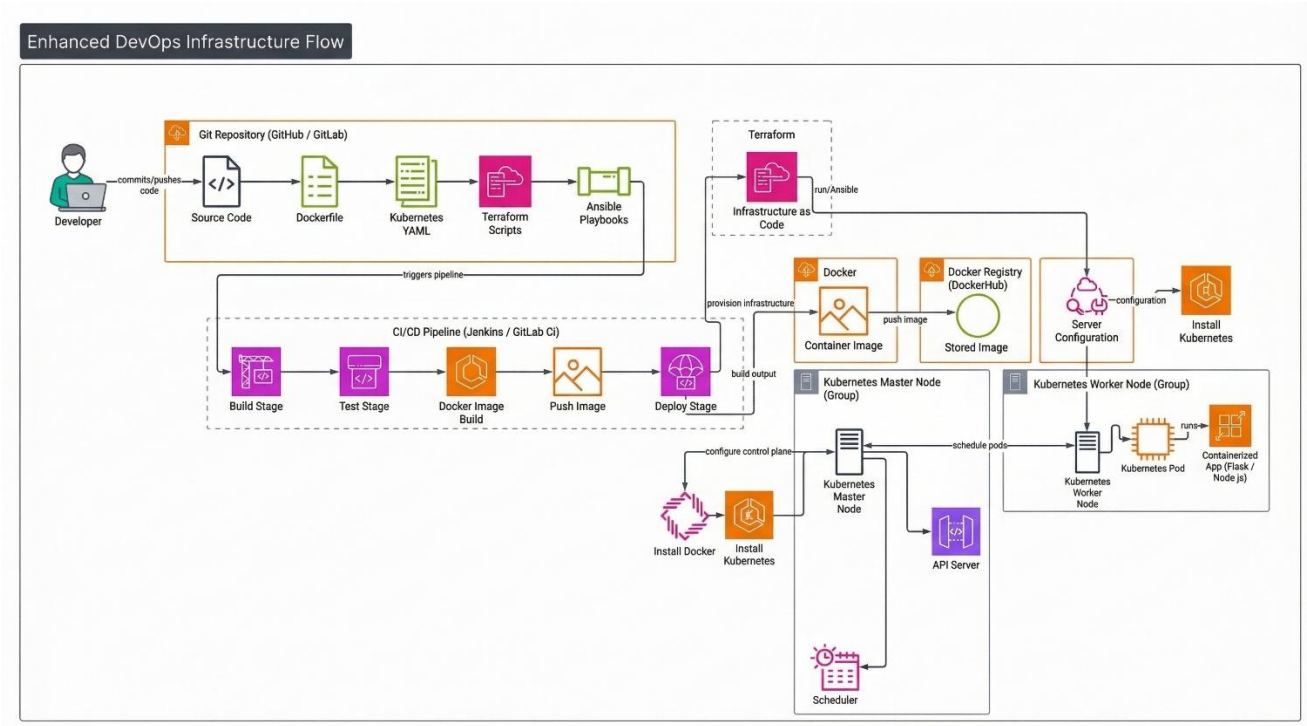

Figure 1 : Architecture globale du système

Description de l'architecture

L'architecture mise en place repose sur une chaîne DevOps permettant d'automatiser l'ensemble du processus de déploiement.

Le processus commence par le développeur qui pousse le code source vers un dépôt GitHub. Ce dépôt contient l'ensemble des éléments nécessaires, notamment le code de l'application de réservation d'hôtel, le Dockerfile, les fichiers de configuration Kubernetes ainsi que les scripts Terraform et les playbooks Ansible.

Un pipeline CI/CD basé sur GitHub Actions est déclenché automatiquement après chaque modification du code. Ce pipeline permet d'automatiser les différentes étapes, notamment :

- La construction (build) de l'application
- Les tests éventuels
- La création de l'image Docker
- Le déploiement de l'application sur le cluster Kubernetes

L'infrastructure est provisionnée sur AWS à l'aide de Terraform, qui permet de créer automatiquement les machines virtuelles (instances EC2), le réseau et les règles de sécurité.

Ensuite, Ansible est utilisé pour configurer les serveurs en installant Docker et Kubernetes, et en préparant le cluster.

Le cluster Kubernetes est composé de deux nœuds :

- Un nœud **Master**, responsable de la gestion du cluster (API Server, Scheduler, Controller)
- Un nœud **Worker**, chargé d'exécuter les applications sous forme de pods

Les applications sont conteneurisées avec Docker et déployées dans Kubernetes, garantissant ainsi leur portabilité et leur scalabilité.

2.3 Technologies utilisées

2.3.1 AWS (Amazon Web Services)

AWS est utilisé comme plateforme cloud pour héberger l'infrastructure du projet.

Il permet de créer des machines virtuelles (EC2), de configurer le réseau (VPC, Subnet) et de gérer les règles de sécurité. Cette solution offre une grande flexibilité et permet de déployer rapidement des environnements à la demande.

2.3.2 Terraform

Terraform est un outil d'Infrastructure as Code (IaC) qui permet d'automatiser la création des ressources cloud.

Dans ce projet, il est utilisé pour :

- Créer le réseau (VPC, Subnet)
- Provisionner les instances EC2
- Configurer les groupes de sécurité

Il permet de reproduire l'infrastructure de manière fiable et automatisée.

2.3.3 Ansible

Ansible est un outil d'automatisation utilisé pour configurer les serveurs.

Dans ce projet, il permet :

- D'installer Docker
- D'installer Kubernetes
- D'initialiser le nœud Master
- De connecter le nœud Worker

Les configurations sont définies sous forme de playbooks exécutés automatiquement.

2.3.4 Docker

Docker est utilisé pour la conteneurisation de l'application.

Il permet de créer des images contenant l'application et toutes ses dépendances, garantissant ainsi sa portabilité et sa compatibilité avec différents environnements.

2.3.5 Kubernetes

Kubernetes est une plateforme d'orchestration de conteneurs.

Il permet de :

- Déployer les applications
- Gérer les pods
- Assurer la scalabilité
- Maintenir la disponibilité des services

Le cluster mis en place est composé d'un nœud Master et d'un nœud Worker.

2.3.6 GitHub Actions (CI/CD)

GitHub Actions est utilisé pour mettre en place un pipeline CI/CD.

Il permet d'automatiser :

- Le build de l'application
- La création des images Docker
- Le déploiement

Cela réduit les interventions manuelles et améliore la rapidité du déploiement.

2.4 Sécurité et accès au système

2.4.1 Accès SSH

L'accès aux machines est sécurisé grâce à une clé SSH.

```
malak@Malak:/mnt/c/Users/malak/Desktop/k8s-cluster-starter$ ssh -i ~/.ssh/k8s-key ec2-user@63.33.61.65 \
  "sudo cp /etc/kubernetes/admin.conf /tmp/admin.conf && sudo chmod 644 /tmp/admin.conf"
```

Figure 2 : Clé SSH

Cela permet une connexion sécurisée aux serveurs pour effectuer les configurations nécessaires.

2.4.2 Accès au Kubernetes Dashboard

Un Kubernetes Dashboard a été déployé pour superviser le cluster.

L'accès à ce dashboard est sécurisé via un token d'authentification généré à l'aide de la commande suivante :

```
[ec2-user@ip-10-0-1-84 ~]$ kubectl get secret admin-user-token -n kubernetes-dashboard \
>   -o jsonpath='{.data.token}' | base64 --decode
eyJhbGciOiJSUzI1NiIsImtpZCI6IlZIVktpWEExYUt4TGxjT29maTFzZFhqTURqNVVJWU9KYnBtZWNtbkg3dmMifQ.eyJpc3MiOiJrdWJlcm5ldGVzL3NlcnZpY2VhY2NvdW50
Iiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9uYW1lc3BhY2UiOiJrdWJlcm5ldGVzLWRhc2hib2FyZCIsImt1YmVybmV0ZXMuaW8vc2VydmljZWFjY291bnQvc2VjcmV
0Lm5hbWUiOiJhZG1pbi11c2VyLXRva2VuIiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9zZXJ2aWNlLWFjY291bnQubmFtZSI6ImFkbWluLXVzZXIiLCJrdWJlcm5ldG
VzLmlvL3NlcnZpY2VhY2NvdW50L3NlcnZpY2UtYWNjb3VudC51aWQiOiIxMzgxMTRiOC02YmMyLTQ0ZmEtYTc0ZC1mMTIyMjI0NjBiMjQiLCJzdWIiOiJzeXN0ZW06c2VydmljZ
WFjY291bnQ6a3ViZXJuZXRlcy1kYXNoYm9hcmQ6YWRtaW4tdXNlciJ9.ku8QubyJru3HY0OpZvW-1zSjWZQB0aNGRtDIvCm3vSEfr4ArKEI8GSidm8W7cvDz3CZJROd6v8ce3TF
kBCv1j1HYU7YXsRXvMspXCmgz_P19wN6vEsLfSRdaCgNxOyPsxA8z75rzVR_6yAfX8VCniatcTq-q-5am01glW0ukL7259KdEgy2AokUmKc6zCCUpsNEt3vd1YRxPUwc8JXJFSW
[ec2-user@ip-10-0-1-84 ~]$
```

Figure 3 : token d'authentification

Le dashboard est accessible via un proxy local :

```
malak@Malak:/mnt/c/Users/malak/Desktop/k8s-cluster-starter$ kubectl --kubeconfig ~/.kube/k8s-config proxy
Starting to serve on 127.0.0.1:8001
E0317 18:02:56.322218    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.347928    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.354872    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.364576    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.365810    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.386228    3759 proxy_server.go:147] Error while proxying request: context canceled
```

Figure 4 : Commande pour accéder au Dashboard

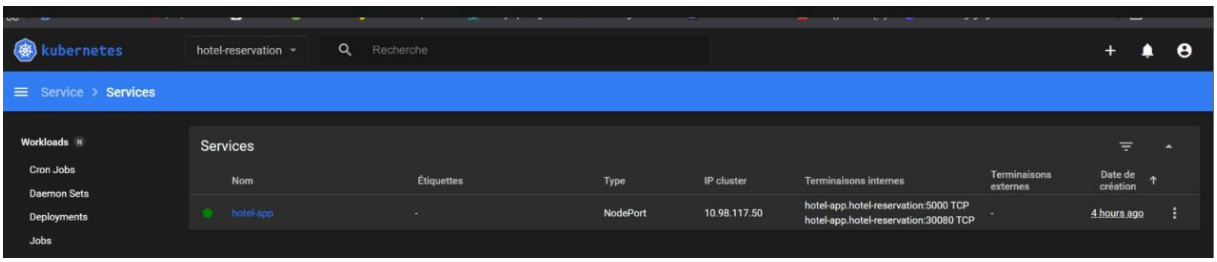

Figure 5 : Kubernetes Dashboard

2.5 Conclusion

Dans ce chapitre, nous avons présenté l'architecture globale du système ainsi que les technologies utilisées dans le cadre de ce projet. Cette architecture repose sur une approche DevOps intégrant l'automatisation, la conteneurisation et l'orchestration des applications.

Le chapitre suivant sera consacré à la mise en œuvre du projet, en détaillant les différentes étapes de déploiement et de configuration.

Chapitre 3 : Mise en œuvre du projet

3.1 Introduction

Dans ce chapitre, nous présentons les différentes étapes de mise en œuvre du projet. Il s'agit de détailler le processus suivi pour déployer l'application de réservation d'hôtel, depuis la création de l'infrastructure jusqu'au déploiement final sur Kubernetes.

3.2 Mise en place de l'environnement AWS

Avant de commencer le déploiement, il est nécessaire de configurer l'environnement AWS afin de pouvoir créer et gérer les ressources cloud.

Configuration des accès AWS

L'accès à AWS est réalisé via l'AWS CLI, configurée avec les identifiants suivants :

aws configure

Cette commande permet de renseigner :

- AWS Access Key
- AWS Secret Key
- Région (ex : eu-west-1)

Cette étape est indispensable pour permettre à Terraform de communiquer avec AWS.

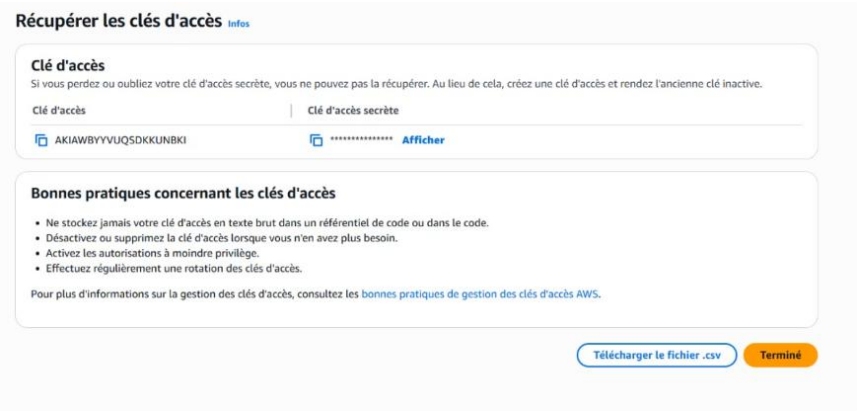

Figure 6 : Clé d'accès

Rôle d'AWS dans le projet

AWS est utilisé pour héberger toute l'infrastructure du projet.

Il permet de créer :

- Des machines virtuelles (EC2)
- Un réseau virtuel (VPC et Subnet)
- Des règles de sécurité (Security Groups)

Dans ce projet, deux instances EC2 sont utilisées :

- Une pour le **Master Kubernetes**
- Une pour le **Worker Kubernetes**

3.3 Mise en place de l'infrastructure avec Terraform

Une fois AWS configuré, Terraform est utilisé pour automatiser la création de l'infrastructure.

Initialisation du projet

```
PS C:\Users\malak\Desktop\k8s-cluster-starter\terraform> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Figure 7 : Commande d'initialisations de terraform

Permet de :

- Télécharger les plugins AWS
- Préparer le projet

Planification

```
malak@Malak MINGW64 ~/Desktop/k8s-cluster-starter/terraform
$ terraform plan
data.aws_ami.amazon_linux_2: Reading...
data.aws_ami.amazon_linux_2: Read complete after 1s [id=ami-08ae1035a3a101a88]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbo
ls:
  + create

Terraform will perform the following actions:

  # aws_instance.k8s_master will be created
  + resource "aws_instance" "k8s_master" {
      + ami                                  = "ami-08ae1035a3a101a88"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
```

Figure 8 : Commande de Planification

Permet de :

- Visualiser les ressources à créer
- Vérifier la configuration

Création de l'infrastructure

```
malak@Malak MINGW64 ~/Desktop/k8s-cluster-starter/terraform
$ terraform apply
data.aws_ami.amazon_linux_2: Reading...
data.aws_ami.amazon_linux_2: Read complete after 1s [id=ami-08ae1035a3a101a88]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbo
ls:
  + create

Terraform will perform the following actions:

  # aws_instance.k8s_master will be created
  + resource "aws_instance" "k8s_master" {
      + ami                                  = "ami-08ae1035a3a101a88"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
```

Figure 9 : Commande pour la Création de l'infrastructure

Cette commande crée automatiquement :

- VPC (10.0.0.0/16)
- Subnet (10.0.1.0/24)
- Internet Gateway
- Security Group (ports ouverts : 22, 6443, 30080, 30500, 30090, 30030)
- 2 instances EC2 (Master + Worker)

Après exécution, Terraform fournit :

- L'IP publique du Master
- L'IP publique du Worker

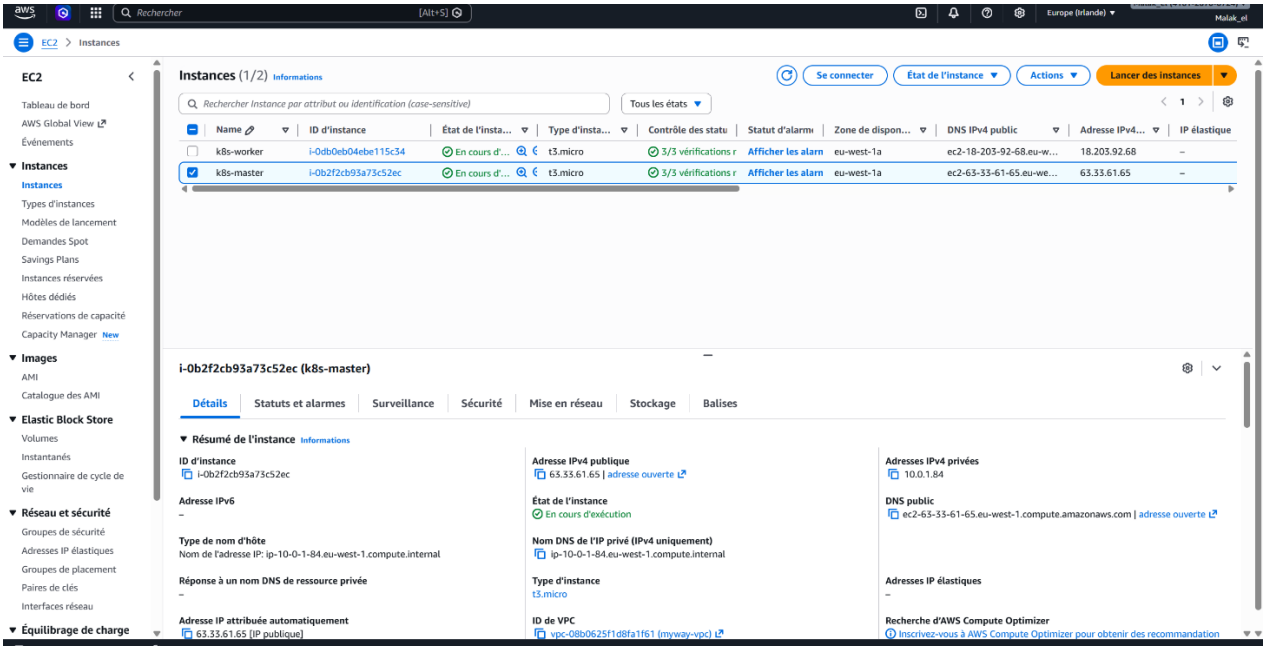

Figure 10 : Les deux instance créer

```
malak@Malak MINGW64 ~/Desktop/k8s-cluster-starter/terraform
$ terraform output
backend_url = "http://63.33.61.65:30500"
frontend_url = "http://63.33.61.65:30080"
grafana_url = "http://63.33.61.65:30030"
health_check_url = "http://63.33.61.65:30500/api/health"
master_instance_id = "i-0b2f2cb93a73c52ec"
master_private_ip = "10.0.1.84"
master_public_ip = "63.33.61.65"
prometheus_url = "http://63.33.61.65:30090"
ssh_command_master = "ssh -i ../ssh/k8s-key ec2-user@63.33.61.65"
ssh_command_worker = "ssh -i ../ssh/k8s-key ec2-user@18.203.92.68"
worker_instance_id = "i-0db0eb04ebe115c34"
worker_private_ip = "10.0.1.56"
worker_public_ip = "18.203.92.68"
```

Figure 11 : Terraform output

3.4 Configuration des serveurs avec Ansible

Après la création de l'infrastructure, Ansible est utilisé pour configurer automatiquement les machines.

- **Mise à jour de l’inventaire**

Le fichier inventory.ini est mis à jour avec :

- IP du Master
- IP du Worker

```
[master]
k8s-master ansible_host=63.33.61.65 ansible_user=ec2-user

[workers]
k8s-worker ansible_host=18.203.92.68 ansible_user=ec2-user
```

Figure 12 : Mise à jour de l'inventaire

- **Test de connexion**

```
malak@Malak:/mnt/c/Users/malak/Desktop/k8s-cluster-starter/ansible$ ansible all -i inventory.ini -m ping
[WARNING]: Platform linux on host k8s-master is using the discovered Python interpreter at /usr/bin/python3.7, but future installation
of another Python interpreter could
change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for mor
e information.
k8s-master | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.7"
    },
    "changed": false,
    "ping": "pong"
}
[WARNING]: Platform linux on host k8s-worker is using the discovered Python interpreter at /usr/bin/python3.7, but future installation
of another Python interpreter could
change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for mor
e information.
k8s-worker | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.7"
    },
    "changed": false,
    "ping": "pong"
}
```

Figure 13 : Test de connectivité

Vérifie que les machines sont accessibles.

- **Installation des composants**

Les playbooks sont exécutés dans l’ordre suivant :

1- Installation de Docker

```
malak@Malak:/mnt/c/Users/malak/Desktop/k8s-cluster-starter/ansible$ ansible-playbook playbooks/01-install-docker.yml

PLAY [Installer Docker sur tous les noeuds Kubernetes] ***************************************************************************
****************************************

TASK [Gathering Facts] ***********************************************************************************************************
****************************************
[WARNING]: Platform linux on host k8s-master is using the discovered Python interpreter at /usr/bin/python3.7, but future installation
of another Python interpreter could
change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for mor
e information.
ok: [k8s-master]
[WARNING]: Platform linux on host k8s-worker is using the discovered Python interpreter at /usr/bin/python3.7, but future installation
of another Python interpreter could
change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for mor
e information.
ok: [k8s-worker]
```

Figure 14 : Installation de Docker

2- Installation de Kubernetes

```
# Install kubeadm, kubelet, kubectl on both nodes
ansible-playbook playbooks/02-install-kubernetes.yml
```

Figure 15 : Installation de Kubernetes

3- Initialisation du Master

```
# Initialize the master (kubeadm init + Flannel CNI)
ansible-playbook playbooks/03-init-master.yml
```

Figure 16 : Initialisation du Master

4- Ajout du Worker

```
malak@Malak:/mnt/c/Users/malak/Desktop/k8s-cluster-starter/ansible$ ansible-playbook playbooks/04-join-worker.yml

PLAY [Joindre le Worker au Cluster Kubernetes] ***********************************************************************************
****************************************

TASK [Gathering Facts] ***********************************************************************************************************
****************************************
[WARNING]: Platform linux on host k8s-worker is using the discovered Python interpreter at /usr/bin/python3.7, but future installation
of another Python interpreter could
change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for mor
e information.
ok: [k8s-worker]
```

Figure 17 : Ajout du Worker

3.5 Accès sécurisé via SSH

L'accès aux différentes machines virtuelles déployées sur AWS EC2 est assuré via le protocole SSH (Secure Shell). Ce protocole permet d'établir une connexion distante sécurisée entre le poste local de l'utilisateur et les instances cloud.

Lors de la création de l'infrastructure, une clé SSH est générée automatiquement. Cette clé est utilisée pour authentifier l'utilisateur sans nécessiter de mot de passe, renforçant ainsi la sécurité du système.

La connexion aux instances se fait à l'aide de la commande suivante :

```
malak@Malak:/mnt/c/Users/malak/Desktop/k8s-cluster-starter$ ssh -i ~/.ssh/k8s-key ec2-user@63.33.61.65 \
  "sudo cp /etc/kubernetes/admin.conf /tmp/admin.conf && sudo chmod 644 /tmp/admin.conf"
```

Figure 18 : Accès sécurisé via SSH

où :

- ssh/k8s-key représente le chemin vers la clé privée
- ec2-user est l'utilisateur par défaut des instances EC2 (Amazon Linux)
- IP correspond à l'adresse IP publique de la machine cible

3.6 Vérification du cluster Kubernetes

Une fois le cluster Kubernetes correctement configuré et initialisé, il est essentiel de vérifier son bon fonctionnement ainsi que l'état des différents nœuds qui le composent. Pour cela, la commande suivante est utilisée :

```
malak@Malak:/mnt/c/Users/malak/Desktop/k8s-cluster-starter$ kubectl --kubeconfig ~/.kube/k8s-config get nodes
NAME                                        STATUS   ROLES           AGE   VERSION
ip-10-0-1-56.eu-west-1.compute.internal     Ready    <none>          16m   v1.29.0
ip-10-0-1-84.eu-west-1.compute.internal     Ready    control-plane   17m   v1.29.0
```

Figure 19 : Vérification du clusters

3.7 Configuration de kubectl (kubeconfig)

Pour accéder au cluster Kubernetes depuis l'environnement local (WSL), le fichier **admin.conf** généré sur le nœud master a été récupéré puis copié dans le répertoire **~/.kube/config**. Ce fichier permet à **kubectl** de se connecter au cluster en utilisant les informations de sécurité et de configuration nécessaires.

Ensuite, l'adresse du serveur a été modifiée afin d'utiliser l'IP publique de la machine EC2, ce qui permet un accès distant au cluster.

Grâce à cette configuration, il est possible d'exécuter des commandes kubectl et de gérer le cluster à distance. Le lancement du proxy avec la commande :

```
malak@Malak:/mnt/c/Users/malak/Desktop/k8s-cluster-starter$ kubectl --kubeconfig ~/.kube/k8s-config proxy
Starting to serve on 127.0.0.1:8001
E0317 18:02:56.322218    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.347928    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.354872    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.364576    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.365810    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.386228    3759 proxy_server.go:147] Error while proxying request: context canceled
```

Figure 20 : Kubeconfig

Confirme que la communication avec le cluster est opérationnelle. Les messages *"context canceled"* observés ne sont pas bloquants.

3.8 Kubernetes Dashboard et authentification

Le **Kubernetes Dashboard** est une interface web permettant de superviser et gérer visuellement le cluster (pods, services, déploiements, etc.), ce qui facilite grandement l'administration par rapport à la ligne de commande.

3.8.1 Déploiement

Le Dashboard est installé via le manifest officiel :

```
[ec2-user@ip-10-0-1-84 ~]$ kubectl apply -f https://raw.githubusercontent.com/kubernetes/dashboard/v2.7.0/aio/deploy/recommended.yaml
namespace/kubernetes-dashboard created
serviceaccount/kubernetes-dashboard created
service/kubernetes-dashboard created
secret/kubernetes-dashboard-certs created
secret/kubernetes-dashboard-csrf created
secret/kubernetes-dashboard-key-holder created
configmap/kubernetes-dashboard-settings created
role.rbac.authorization.k8s.io/kubernetes-dashboard created
clusterrole.rbac.authorization.k8s.io/kubernetes-dashboard created
rolebinding.rbac.authorization.k8s.io/kubernetes-dashboard created
clusterrolebinding.rbac.authorization.k8s.io/kubernetes-dashboard created
deployment.apps/kubernetes-dashboard created
service/dashboard-metrics-scraper created
deployment.apps/dashboard-metrics-scraper created
```

Figure 21 : Déploiement

3.8.2 Création de l'utilisateur admin + récupération du token

Un utilisateur administrateur est créé afin d'obtenir un accès sécurisé. Le token d'authentification est récupéré avec :

```
[ec2-user@ip-10-0-1-84 ~]$ kubectl get secret admin-user-token -n kubernetes-dashboard \
>   -o jsonpath='{.data.token}' | base64 --decode
eyJhbGciOiJSUzI1NiIsImtpZCI6IlZIVktpWEExYUt4TGxjT29maTFzZFhqTURqNVVJWU9KYnBtZWNtbkg3dmMifQ.eyJpc3MiOiJrdWJlcm5ldGVzL3NlcnZpY2VhY2NvdW50
Iiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9uYW1lc3BhY2UiOiJrdWJlcm5ldGVzLWRhc2hib2FyZCIsImt1YmVybmV0ZXMuaW8vc2VydmljZWFjY291bnQvc2VjcmV
0Lm5hbWUiOiJhZG1pbi11c2VyLXRva2VuIiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9zZXJ2aWNlLWFjY291bnQubmFtZSI6ImFkbWluLXVzZXIiLCJrdWJlcm5ldG
VzLmlvL3NlcnZpY2VhY2NvdW50L3NlcnZpY2UtYWNjb3VudC51aWQiOiIxMzgxMTRiOC02YmMyLTQ0ZmEtYTc0ZC1mMTIyMjI0NjBiMjQiLCJzdWIiOiJzeXN0ZW06c2Vydmljz
WFjY291bnQ6a3ViZXJuZXRlcy1kYXNoYm9hcmQ6YWRtaW4tdXNlciJ9.ku8QubyJru3HY0OpZvW-1zSjWZQB0aNGRtDIvCm3vSEfr4ArKEI8GSidm8W7cvDz3CZJROd6v8ce3TF
kBCv1j1HYU7YXsRXvMspXCmgz_P19wN6vEsLfSRdaCgNxOyPsxA8z75rzVR_6yAfX8VCniatcTq-q-5am01glW0ukL7259KdEgy2AokUmKc6zCCUpsNEt3vd1YRxPUwc8JXJFSW
[ec2-user@ip-10-0-1-84 ~]$
```

Figure 22 : Token d'authentification

3.8.3 Accès au Dashboard

Le Dashboard est accessible localement via :

```
malak@Malak:/mnt/c/Users/malak/Desktop/k8s-cluster-starter$ kubectl --kubeconfig ~/.kube/k8s-config proxy
Starting to serve on 127.0.0.1:8001
E0317 18:02:56.322218    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.347928    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.354872    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.364576    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.365810    3759 proxy_server.go:147] Error while proxying request: context canceled
E0317 18:02:56.386228    3759 proxy_server.go:147] Error while proxying request: context canceled
```

Figure 23 : Accès Dashboard

Puis en accédant à l'URL fournie dans le navigateur.

Cette approche via kubectl proxy assure une **connexion sécurisée** sans exposition directe du service sur Internet.

3.9 Conteneurisation avec Docker

L'application a été conteneurisée à l'aide de **Docker**, ce qui permet d'encapsuler tout l'environnement (code, dépendances, configurations) dans une image portable et reproductible.

Construction de l'image Docker

L'image est créée à partir du fichier Dockerfile avec la commande :

```
PS C:\Users\malak\Desktop\k8s-cluster-starter> docker build -t hotel-app .
```

Figure 24 : Création de l'image

Gestion des images

Après la construction, les images disponibles peuvent être vérifiées avec :

```
PS C:\Users\malak\Desktop\k8s-cluster-starter> docker build -t hotel-app .
```

Figure 25 : Gestion des images

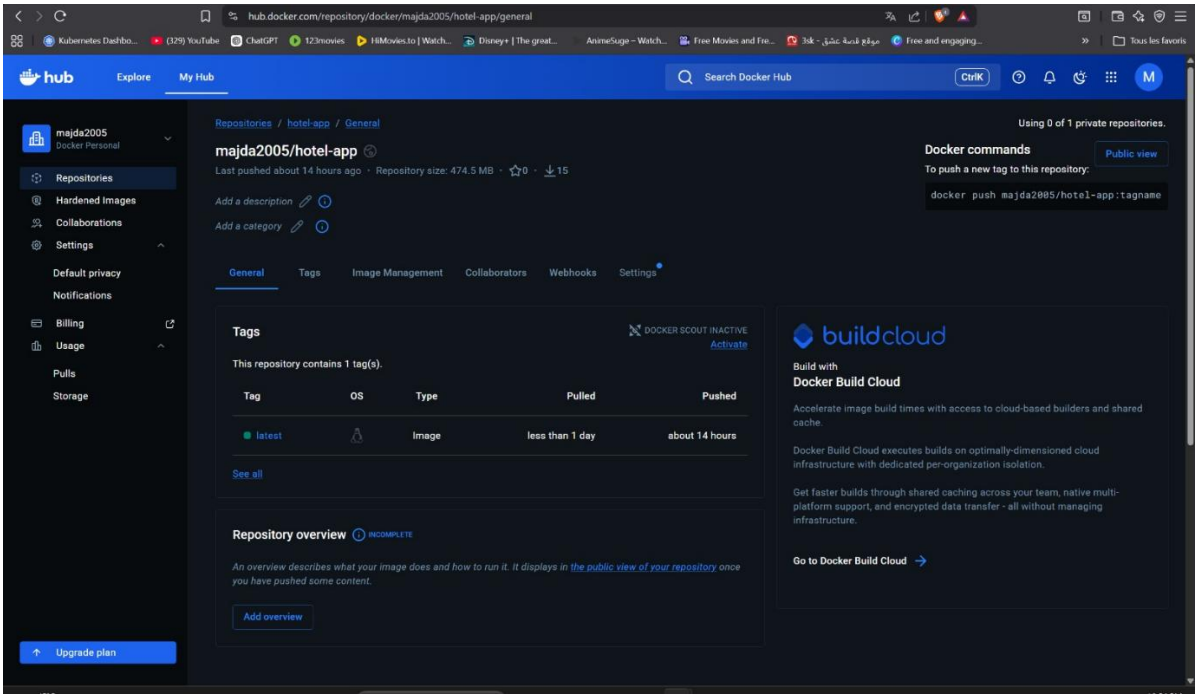

Figure 26 : L'image dans Docker

La conteneurisation permet :

- De **packager l'application** avec toutes ses dépendances
- De garantir une **portabilité** entre différents environnements
- De **faciliter le déploiement** sur Kubernetes

3.10 Gestion du code source avec GitHub:

Le code source du projet est hébergé sur GitHub afin de faciliter la gestion de version, la collaboration et l'intégration continue.

Cela permet :

- Le suivi des modifications
- La sauvegarde du projet
- L'intégration avec GitHub Actions pour le CI/CD

1. Clone du projet

```
malak@Malak MINGW64 ~/Desktop/k8s-cluster-starter
$ git clone https://github.com/Malak323-e/Hotel_reservations.git
Cloning into 'Hotel_reservations'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

Figure 27: Clonage du repository GitHub du projet sur la machine locale.

2. Structure du repository

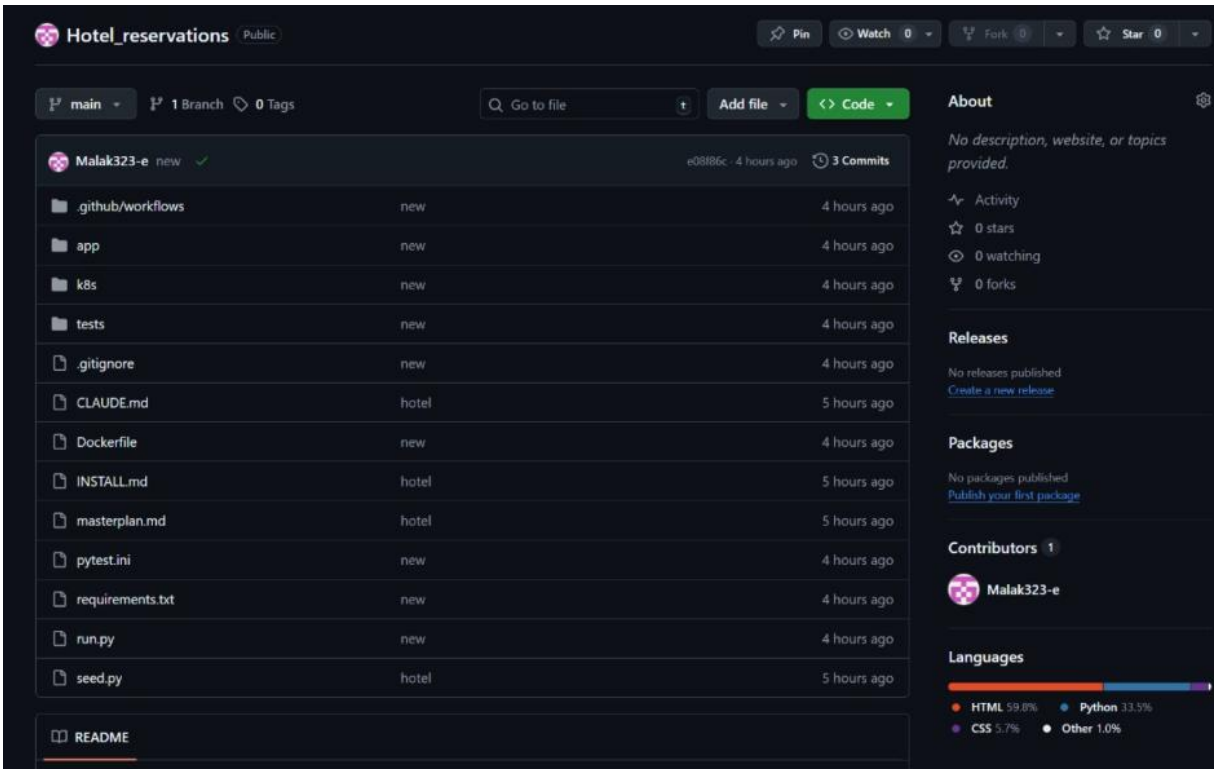

Figure 28 : Structure du repository GitHub contenant les fichiers de l'application, les configurations Docker, Kubernetes et le pipeline CI/CD.

3.11 Pipeline CI/CD avec GitHub Actions

Un pipeline CI/CD a été mis en place avec GitHub Actions afin d'automatiser les différentes étapes du déploiement et garantir une intégration continue du projet.

Fonctionnement :
Le pipeline est déclenché automatiquement après chaque push sur le repository GitHub, puis exécute les étapes suivantes :

- Build de l'application
- Création de l'image Docker
- Push de l'image vers un registre (optionnel)
- Déploiement automatique sur le cluster Kubernetes

Cette automatisation permet :

- De réduire les erreurs humaines
- D'assurer un déploiement rapide et fiable
- De maintenir une cohérence entre les versions

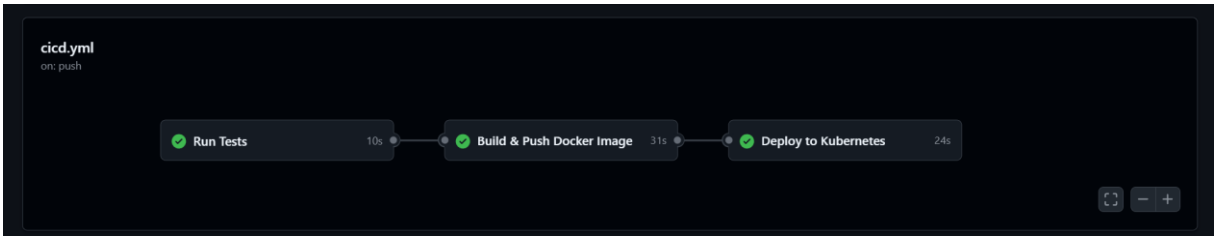

Figure 29: Exécution du pipeline GitHub Actions

3.12 Déploiement de l'application

Après le déploiement, Kubernetes crée automatiquement les ressources nécessaires :

- Les **pods** contenant l'application
- Les **services** permettant d'exposer l'application

On peut vérifier le bon fonctionnement avec :

```
[ec2-user@ip-10-0-1-84 ~]$ kubectl get nodes
NAME                                      STATUS   ROLES           AGE     VERSION
ip-10-0-1-56.eu-west-1.compute.internal   Ready    <none>          7m24s   v1.29.0
ip-10-0-1-84.eu-west-1.compute.internal   Ready    control-plane   8m39s   v1.29.0
[ec2-user@ip-10-0-1-84 ~]$ kubectl get pods -A
NAMESPACE      NAME                                                       READY   STATUS    RESTARTS   AGE
kube-flannel   kube-flannel-ds-g8bcv                                      1/1     Running   0          8m39s
kube-flannel   kube-flannel-ds-zlkm7                                      1/1     Running   0          7m41s
kube-system    coredns-76f75df574-lj8n4                                   1/1     Running   0          8m38s
kube-system    coredns-76f75df574-mctdr                                   1/1     Running   0          8m38s
kube-system    etcd-ip-10-0-1-84.eu-west-1.compute.internal               1/1     Running   0          8m54s
kube-system    kube-apiserver-ip-10-0-1-84.eu-west-1.compute.internal     1/1     Running   0          8m52s
kube-system    kube-controller-manager-ip-10-0-1-84.eu-west-1.compute.internal 1/1 Running 0          8m52s
kube-system    kube-proxy-h22d4                                           1/1     Running   0          8m39s
kube-system    kube-proxy-zs7pz                                           1/1     Running   0          7m41s
kube-system    kube-scheduler-ip-10-0-1-84.eu-west-1.compute.internal     1/1     Running   0          8m54s
```

Figure 30 : État des nodes et des pods du cluster Kubernetes.

Cette étape permet de rendre l'application accessible, scalable et correctement orchestrée sur le cluster Kubernetes.

3.13 Accès à l'application

L'application est accessible via le navigateur en utilisant l'adresse suivante :

http://63.33.61.65:30080/

- Cette URL correspond au service Kubernetes de type NodePort, permettant d'exposer l'application à l'extérieur du cluster.
- Elle permet aux utilisateurs d'accéder directement à l'application de réservation d'hôtel déployée.

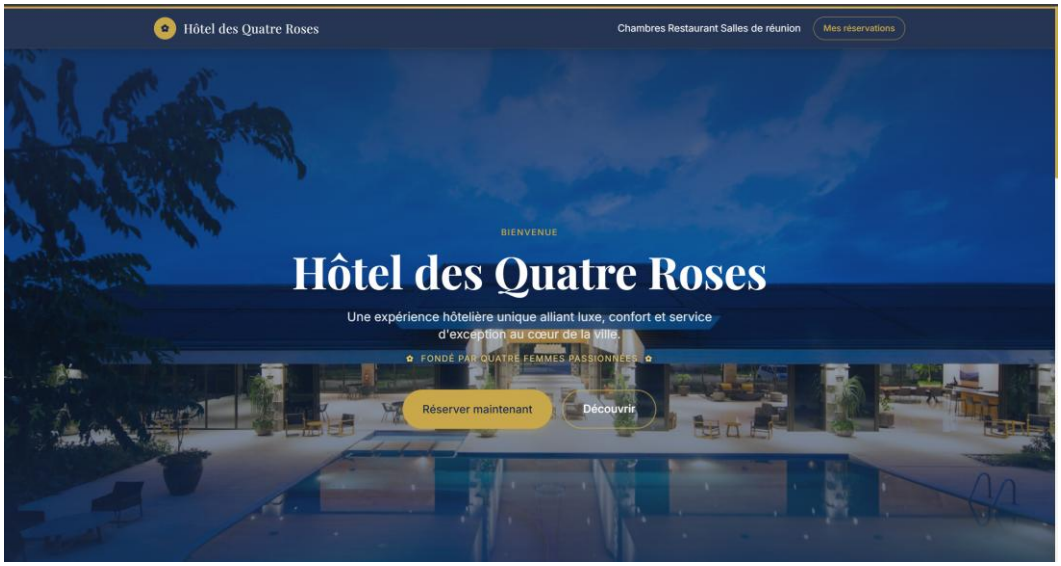

Figure 31: Interface de l'application web

3.14 Conclusion

Ce chapitre a permis de présenter de manière détaillée la mise en œuvre du projet, en mettant en évidence l'ensemble des étapes réalisées, depuis la conteneurisation de l'application jusqu'à son déploiement sur un cluster Kubernetes.

L'intégration des outils Docker, Kubernetes, GitHub Actions et AWS a permis de mettre en place une architecture moderne, automatisée et scalable, répondant aux exigences des environnements cloud actuels.

De plus, l'automatisation via le pipeline CI/CD a garanti un processus de déploiement fiable, rapide et reproductible, tout en réduisant les erreurs humaines et en facilitant la gestion des mises à jour.

Enfin, l'accès à l'application déployée confirme le bon fonctionnement de l'ensemble de l'infrastructure mise en place, validant ainsi une chaîne complète allant du développement jusqu'à la mise en production.

Ce chapitre constitue ainsi une base solide pour l'évaluation du système et l'analyse des performances présentées dans le chapitre suivant.

Chapitre 4 : Présentation et résultats de l'application

4.1 Présentation générale de l'application

L'application développée est un système de réservation d'hôtel intitulé **"Hôtel des Quatre Roses"**. Elle permet aux utilisateurs de consulter les services disponibles et d'effectuer des réservations en ligne de manière simple, rapide et intuitive.

L'interface est moderne, ergonomique et adaptée à une utilisation web, offrant une expérience utilisateur fluide.

- **Page d'accueil :**

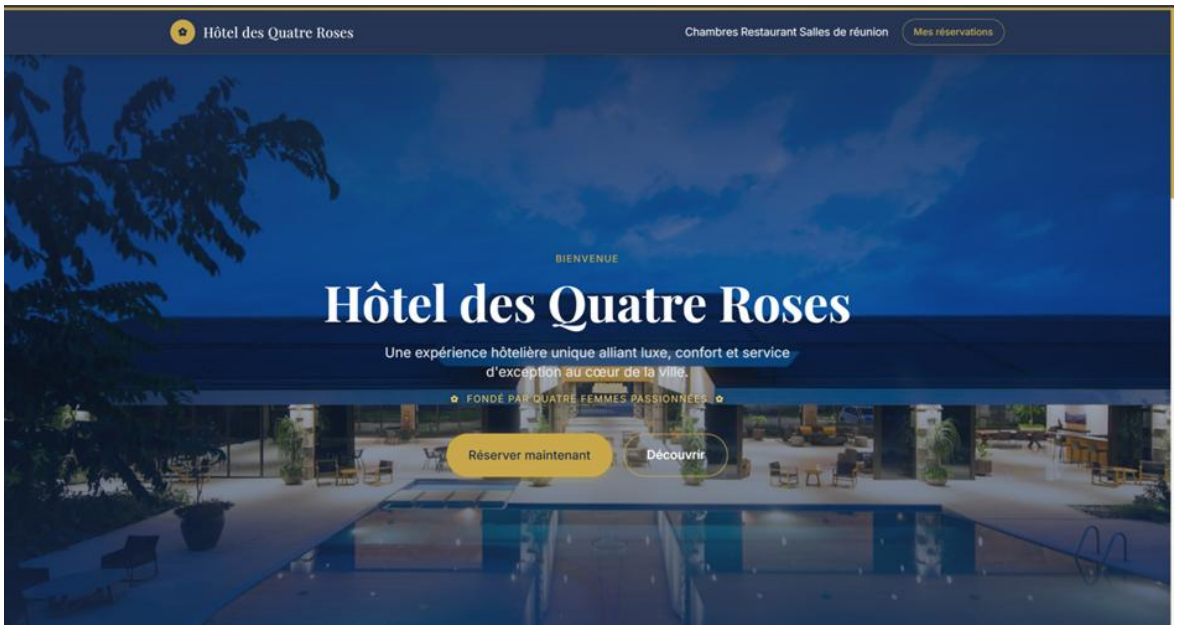

Figure 32 : Page d'acceuil

La page d'accueil présente une interface élégante avec :

- Un message de bienvenue
- Une présentation de l'hôtel
- Un bouton principal **"Réserver maintenant"**

Elle constitue le point d'entrée principal pour l'utilisateur.

- **Présentation des services**

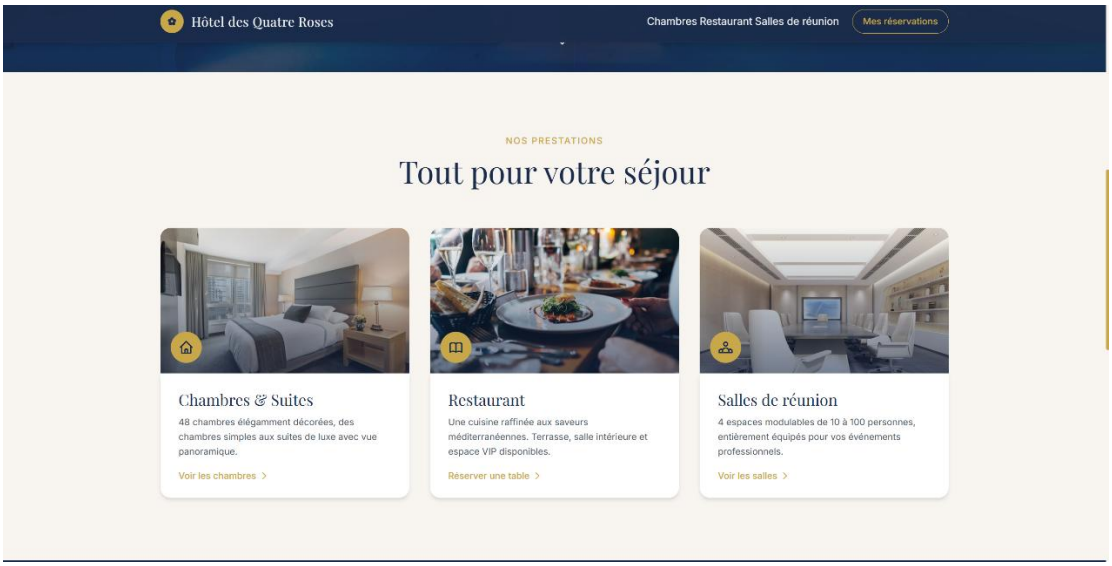

Figure 33 : Présentation des services

L'application propose plusieurs services :

- **Chambres & Suites**
- **Restaurant**
- **Salles de réunion**

Chaque service est présenté sous forme de cartes avec description et accès direct à la réservation.

- **Processus de réservation**

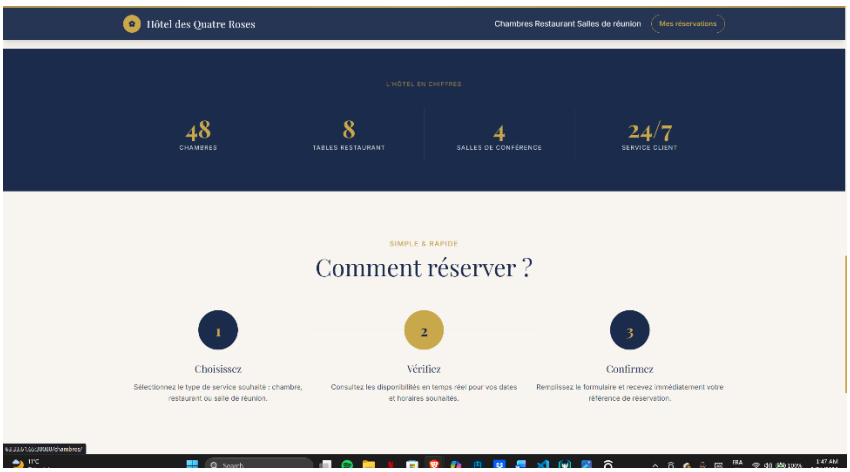

Figure 34 : Processus de réservation

Le processus de réservation est structuré en trois étapes :

1. Choisir le service
2. Vérifier la disponibilité
3. Confirmer la réservation

Cette organisation simplifie l'expérience utilisateur.

- **Réservation**

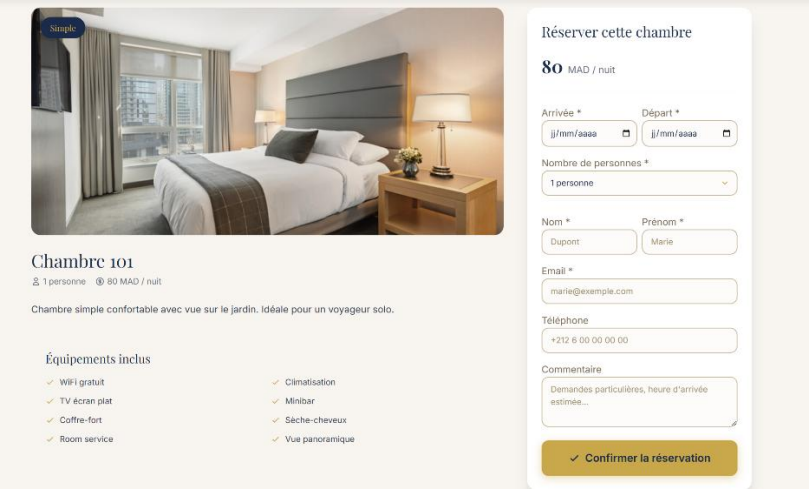

Figure 35 : Formulaire de réservation de chambres

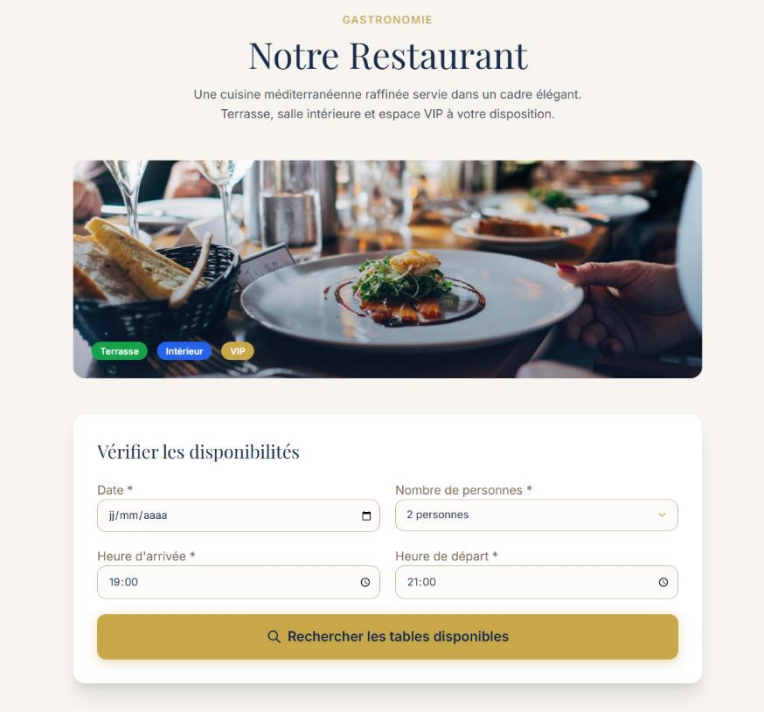

Figure 36 : Formulaire de réservation au Restaurant

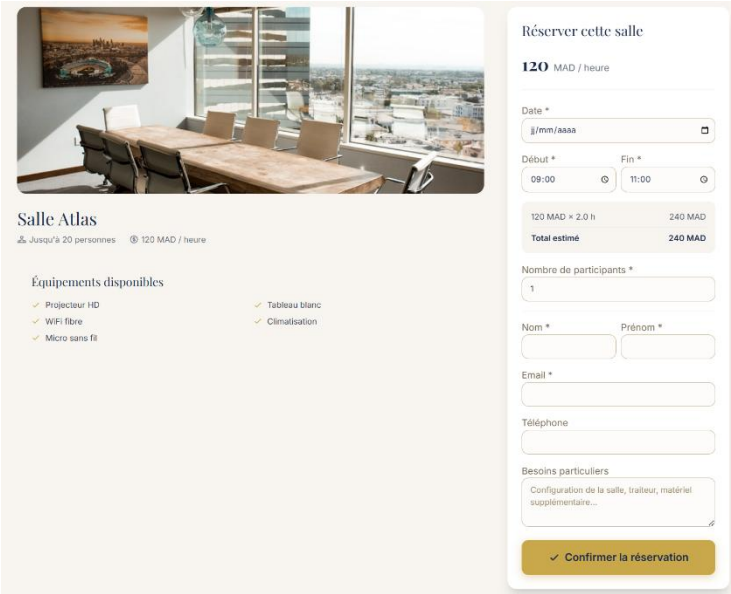

Figure 37: Formulaire de réservation de Salle de réunion

- **Confirmation de réservation**

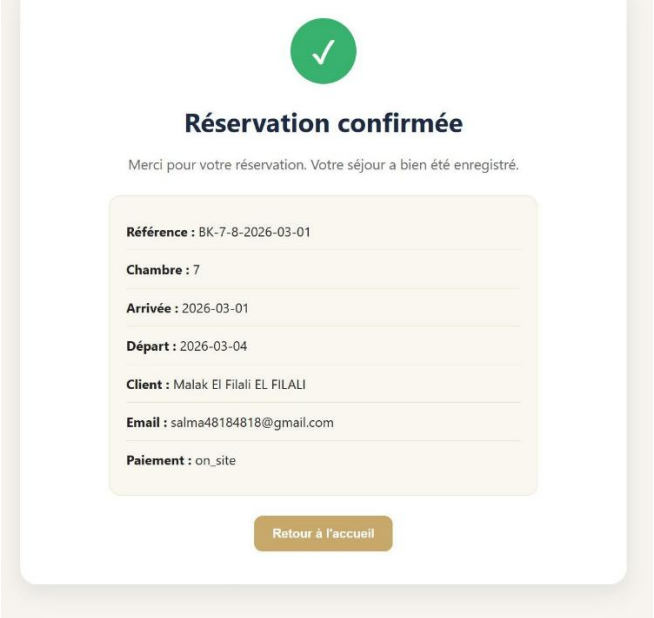

Figure 38 : Confirmation de réservation

- **Stockage des données**

		type_res...	statut	nom_client	prenom_...	email_client	telep
1	8	chambre	confirmee	Dabibe	Anass	anas@gmail.com	"
2	D	chambre	confirmee	Majda	chouiekh	majdack16@gmail.com	06453
3	7	chambre	confirmee	EL FILALI	El Filali	salma48184818@gma...	+2126
4	B	chambre	confirmee	EL FILALI	Malak	salma48184818@gma...	+2126
5							

Figure 39 : Stockage des données

Les réservations sont enregistrées dans une base de données contenant :

- Type de réservation
- Nom et prénom du client
- Email
- Statut de réservation

Cela permet la gestion et le suivi des réservations.

- **Gestion des réservations**

Figure 40 : Réservation des Clients

Chapitre 5 : Analyse des résultats et validation du système

5.1 Introduction

Dans ce chapitre, nous présentons une analyse des résultats obtenus après la mise en œuvre du projet. L'objectif est de valider le bon fonctionnement du système développé, d'évaluer les performances de l'architecture mise en place, et d'identifier les points forts ainsi que les limites du projet.

5.2 Résultats du déploiement Kubernetes

Après la mise en place de l'infrastructure et la configuration du cluster, les différents tests effectués ont permis de confirmer que le cluster Kubernetes est pleinement opérationnel. Les nœuds Master et Worker sont correctement configurés et fonctionnent de manière stable.

Le déploiement des applications sous forme de pods a été réalisé avec succès, et les services sont accessibles sans interruption. Cela démontre que l'orchestration des conteneurs par Kubernetes a été correctement mise en œuvre.

5.3 Validation du fonctionnement de l'application

L'application de réservation d'hôtel a été déployée avec succès sur le cluster Kubernetes et est accessible via le navigateur. Les différentes fonctionnalités ont été testées et validées.

L'utilisateur peut consulter les services proposés, effectuer des réservations de chambres, de restaurant ou de salles de réunion, et recevoir une confirmation de réservation avec une référence unique. Les données saisies sont correctement enregistrées, ce qui confirme le bon fonctionnement du système.

5.4 Supervision et gestion du système

Le déploiement du Kubernetes Dashboard a permis de superviser efficacement le cluster. Grâce à cet outil, il est possible de suivre l'état des pods, des services et des ressources du système.

L'accès sécurisé au dashboard via un token d'authentification garantit la protection des données et du cluster, tout en facilitant la gestion des composants déployés.

5.5 Analyse des performances

Les résultats obtenus mettent en évidence plusieurs points positifs. L'automatisation complète du processus de déploiement, grâce à Terraform et Ansible, permet de réduire considérablement le temps de mise en place de l'infrastructure.

L'utilisation de Docker assure la portabilité de l'application, tandis que Kubernetes garantit une bonne gestion des conteneurs et une disponibilité continue du service. De plus, l'intégration d'un pipeline CI/CD avec GitHub Actions permet d'automatiser les étapes de build et de déploiement, améliorant ainsi l'efficacité du système.

5.6 Limites du système

Malgré les résultats satisfaisants obtenus, certaines limites ont été identifiées. Le cluster mis en place est composé de seulement deux nœuds, ce qui limite la scalabilité et la tolérance aux pannes.

De plus, les aspects de sécurité restent basiques, notamment avec l'ouverture de certains ports et l'absence de mécanismes avancés de protection. Par ailleurs, l'absence d'outils de monitoring tels que Prometheus et Grafana ne permet pas une supervision approfondie des performances du système.

5.7 Perspectives d'amélioration

Afin d'améliorer le système, plusieurs pistes peuvent être envisagées. L'intégration d'outils de monitoring permettrait de mieux analyser les performances du cluster. L'ajout de mécanismes d'auto-scaling améliorerait la gestion des charges.

Il serait également pertinent de renforcer la sécurité en limitant les accès réseau et en mettant en place des politiques de sécurité plus strictes. Enfin, le pipeline CI/CD pourrait être enrichi pour inclure davantage de tests automatisés.

5.8 Conclusion

Ce chapitre a permis d'analyser les résultats obtenus et de valider le bon fonctionnement du système. L'ensemble des tests réalisés confirme que le projet répond aux objectifs fixés, en proposant une solution fonctionnelle, automatisée et basée sur une architecture DevOps moderne.

Perspectives d’évolution

Malgré les résultats satisfaisants obtenus, plusieurs améliorations peuvent être envisagées :

- Ajouter plusieurs nœuds Worker pour améliorer la scalabilité et la tolérance aux pannes
- Intégrer des outils de monitoring comme Prometheus et Grafana
- Renforcer la sécurité (restriction des ports, meilleure authentification)
- Améliorer le pipeline CI/CD avec des tests et un déploiement progressif
- Enrichir l’application (paiement en ligne, comptes utilisateurs, notifications)
- Utiliser une base de données plus performante
- Automatiser davantage le déploiement

Ces améliorations permettraient de rendre le système plus performant, sécurisé et adapté à un environnement réel.

Conclusion générale

Au terme de ce projet, nous avons pu concevoir et mettre en œuvre une solution complète de déploiement d'une application de réservation d'hôtel en s'appuyant sur une approche DevOps moderne. L'ensemble des étapes, allant de la création de l'infrastructure jusqu'au déploiement de l'application, a été réalisé avec succès en utilisant des outils tels que AWS, Terraform, Ansible, Docker, Kubernetes et GitHub Actions.

Ce projet nous a permis de mettre en pratique plusieurs concepts essentiels, notamment l'automatisation de l'infrastructure, la conteneurisation des applications et l'orchestration des services. La mise en place d'un pipeline CI/CD a également permis d'automatiser le processus de déploiement, réduisant ainsi les interventions manuelles et les risques d'erreur.

Les résultats obtenus confirment le bon fonctionnement du système, avec un cluster Kubernetes opérationnel, une application accessible et une gestion efficace des ressources. Ce travail a également permis de développer des compétences techniques solides dans le domaine du Cloud Computing et du DevOps.

Cependant, certaines améliorations peuvent être envisagées, notamment en termes de sécurité, de monitoring et de scalabilité, afin de rendre le système encore plus performant et adapté à un environnement de production.

En conclusion, ce projet constitue une expérience enrichissante qui met en évidence l'importance des pratiques DevOps dans le développement et le déploiement des applications modernes, et ouvre la voie à des perspectives d'évolution intéressantes.

Références

Documentation officielle

- Kubernetes Documentation, *Kubernetes Official Documentation*. Disponible sur : https://kubernetes.io/docs/
- Docker Documentation, *Docker Official Documentation*. Disponible sur : https://docs.docker.com/
- Terraform Documentation, *Terraform by HashiCorp*. Disponible sur : https://developer.hashicorp.com/terraform/docs
- Ansible Documentation, *Ansible Documentation*. Disponible sur : https://docs.ansible.com/
- Amazon Web Services Documentation, *AWS Documentation*. Disponible sur : https://docs.aws.amazon.com/
- GitHub Actions Documentation, *GitHub Actions Documentation*. Disponible sur : https://docs.github.com/en/actions

Articles et ressources utiles

- Red Hat, *What is Kubernetes?* Disponible sur : https://www.redhat.com/en/topics/containers/what-is-kubernetes
- IBM, *What is DevOps?* Disponible sur : https://www.ibm.com/topics/devops
- Microsoft Azure, *CI/CD explained*. Disponible sur : https://learn.microsoft.com/en-us/devops/what-is-ci-cd